

CODELANGID: A CHARACTER-LEVEL CNN FOR PROGRAMMING LANGUAGE IDENTIFICATION

Mohamad Salman | mohamad.salman@mail.utoronto.ca | University of Toronto | Solo project
 Code: <https://github.com/mohamadmsalman82/codelangid>

1 BRIEF PROJECT DESCRIPTION

Syntax highlighters, code search engines, snippet managers and Q&A sites such as Stack Overflow must constantly identify a block of code’s language, often from a short, context-free fragment with no filename or extension (Alreshedy et al., 2018). **This project maps a raw snippet to one of 10 popular languages (Python, Java, C++, JavaScript, Go, C, Ruby, PHP, Rust, C#) from its characters alone.** Extensions and editor metadata are exactly what is missing in pasted snippets, chat messages and scraped pages, and rule-based detectors degrade on short or ambiguous fragments (van Dam & Zaytsev, 2016); GitHub’s Linguist leans on extensions and heuristics (GitHub, 2025), so it says little about a bare snippet—the gap a learned model fills.

Deep learning fits for three reasons. Languages differ by *subtle, overlapping* cues (keywords, operator spellings, bracket and indentation patterns) that a character-level CNN learns automatically rather than by hand-engineering features per language (Zhang et al., 2015). Character input also side-steps a chicken-and-egg problem: a token-based model needs a tokenizer, but choosing one presupposes knowing the language. And the cues are *local* n-gram motifs (`:=`, `#include`)—what small-kernel convolution detects (Kim, 2014). Figure 1 gives the contract: **input** raw text padded/truncated to 256 characters, **output** a softmax over the 10 languages.

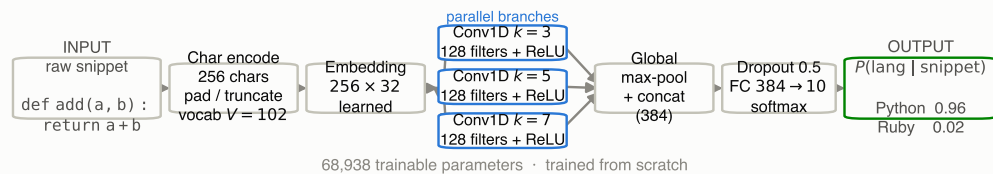


Figure 1: The CodeLangID pipeline. **Input:** a raw snippet, any language, encoded character-by-character into a fixed 256-token window. **Output:** $P(\text{lang} \mid \text{snippet})$ over the 10 classes.

2 INDIVIDUAL CONTRIBUTIONS AND RESPONSIBILITIES

This is a **solo project**: there are no other team members, so every item in Table 1 is my own work and I hold all responsibilities end to end. All code is reproducible (`src/`, seed 360); every number below regenerates from a clean checkout.

Table 1: Work breakdown; all items are my sole responsibility (solo project). Final tuning and the report remain in progress.

Component	Status	Evidence / artefact
Data collection & cleaning	Complete	<code>build_dataset.py</code> : 10,360 snippets (Table 2)
Held-out test collection	Complete	<code>scrape_heldout.py</code> : 740 snippets, 24 repos, 0 leaked
Baseline model	Complete	<code>baseline.py</code> : 90.14% test (Table 3)
Primary model (char-CNN)	Complete	<code>cnn.py</code> : 92.74% test, 82.84% held-out
Alternative model (BiGRU)	Complete	<code>gru.py</code> : tried and rejected (§3.3)
Evaluation & error analysis	Complete	<code>figures.py</code> : Fig. 2, §3.4

3 NOTABLE CONTRIBUTION

My notable contribution is the **complete working pipeline**: a cleaned balanced dataset, a classical baseline, a char-CNN that beats it, and an independently collected held-out set exposing a diagnosable failure mode.

3.1 DATA PROCESSING

Source. Rosetta Code (ros, 2026) (acmeism/RosettaCodeData mirror) implements the same tasks across many languages, so snippets are balanced and labelled by directory structure rather than guessed. Table 2 gives the full pipeline (src/build_dataset.py, seed 360) and its yield, specified so a classmate can reproduce it. Two steps carry most of the weight. *Near-duplicate removal:* Rosetta hosts many near-identical variants of one solution, so MinHash/LSH over 5-character shingles drops pairs with Jaccard ≥ 0.8 (Broder, 1997). *Splitting by task, not by file:* all snippets from a task land in one split—else two “Bubble sort” solutions straddle train and test and inflate accuracy. Each snippet is emitted in **two variants**, raw and nocomment, to test reliance on comment prose versus code structure.

Table 2: Cleaning pipeline and yield (steps 2–4 run during windowing, so the running total is first well-defined at step 4). Result: 10,360 snippets, 1,036 per class (chance = 10%), split 1,162/249/250 tasks.

#	Operation	Removed	Remaining
1	Read 10 language directories	—	18,654 files
2	Window: ≤ 3 line-aligned 256-char windows/file (≥ 3 lines, ≥ 64 char)	1,125 files	
3	Comment/markup filter: drop if comments $> 50\%$ of non-space chars	3,460 win.	
4	Exact dedup: SHA-1 of whitespace-normalized text	327 win.	38,394 snippets
5	Near-dup dedup: MinHash (64 perm, 16 LSH bands), Jaccard ≥ 0.8	353	38,041
6	Balance: cap each class at 1,036 = smallest class (PHP)	27,681	10,360
7	Split by task 70/15/15 (no task spans two splits)	—	7,225 / 1,634 / 1,501
8	Encode: vocab $V=102$ (ASCII + <code><pad></code> + <code><unk></code>), pad/truncate to 256	—	—

Example cleaned training sample (class PYTHON, 173 characters):

```
def mkdirp(path):
    try: os.makedirs(path)
    except OSError as exc: # Python >2.5 <- removed in the 'nocomment' variant
        if exc.errno == errno.EEXIST and os.path.isdir(path): pass
        else: raise
```

Plan for never-before-seen data. Rather than reuse a public dataset I collected the final test set myself (src/scrape_heldout.py): **740 snippets from 24 real GitHub repositories** (e.g. psf/requests, redis/redis, BurntSushi/ripgrep), 74 per language, through the *identical* pipeline. None is a Rosetta source, and it is deliberately a *different distribution*: production library code, not puzzle solutions. Each snippet is checked against the SHA-1 of every training snippet—**0 leaked**—and is never trained or tuned on.

Challenges. (i) PHP is thin on Rosetta (645 usable files vs. Python’s 3,054), so balancing drags *every* class to 1,036—the biggest limit on dataset size. (ii) Splitting by snippet leaked near-identical solutions and inflated early accuracy; splitting by task fixed it. (iii) The regex comment stripper over-strips string literals containing `//` or `#`; acceptable as it is applied uniformly.

3.2 BASELINE MODEL

The baseline uses no deep learning: TF-IDF over **character n-grams** ($n \in [1, 3]$, min_df=2, sub-linear tf, case preserved) into **Multinomial Naive Bayes** on the same splits, mirroring prior work (Khasnabish et al., 2014; Ugurel et al., 2002). It needs no tuning beyond the fixed n-gram range, uses 43,573 features and trains in ≈ 4 s. It is an honest bar, not a straw man—**90.14%** versus 10% chance—so n-grams already carry most of the signal and the CNN must justify itself on what remains.

3.3 PRIMARY MODEL

The primary model (Figure 1) is Embedding(102,32) (padding_idx=0); three *parallel* Conv1d branches of widths 3/5/7, 128 filters each + ReLU; global max-pool per branch; concatenation to 384; Dropout(0.5); Linear(384,10) + softmax. Parallel widths rather than a deep stack see 3-, 5- and 7-character motifs at once—the scale at which fn, := and #include live. It has **68,938 trainable parameters** in 5 weight layers, trained from scratch with cross-entropy and Adam (Kingma & Ba, 2015) (lr 10^{-3} , batch 64, ≤ 30 epochs, early stopping on validation accuracy, patience 6). Training takes **35 s** on an Apple M-series GPU (MPS). Validation loss flattens near epoch 10 and the train/validation gap widens after epoch 12—mild overfitting that early stopping controls: it stopped at epoch 25 with the best validation accuracy at **epoch 19**.

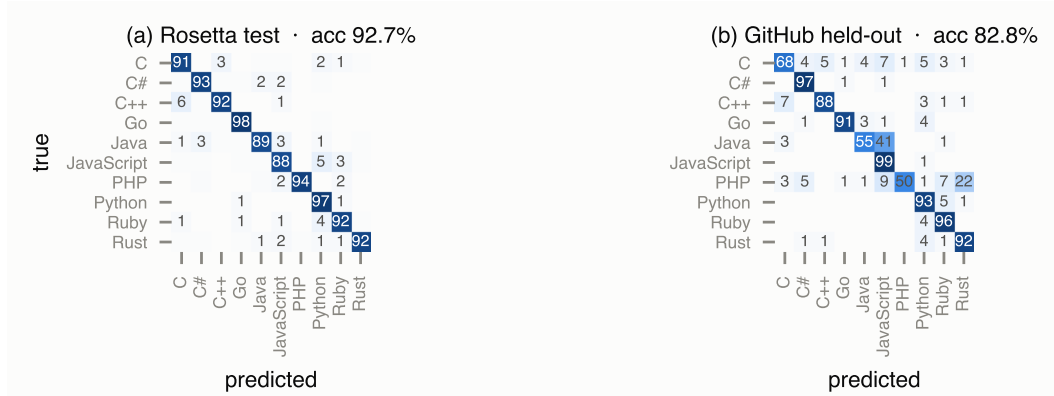


Figure 2: Char-CNN confusion, row-normalized (cell = % of the true class predicted as that column). (a) Rosetta test. (b) Never-before-seen GitHub held-out, where Java→JavaScript and PHP→Rust dominate (§3.4).

I also built the proposal’s bidirectional GRU (`src/gru.py`; embedding 32, hidden 64/direction, max-pooled, 42,186 parameters). **It did not pan out**—the most useful negative result so far: 90.27% test, indistinguishable from Naive Bayes and 2.5 points below the CNN, while taking **647 s to train, 18× the CNN**. Language identity in a 256-character window rests on *local* motifs, so recurrence buys nothing and costs much; the CNN stays primary.

3.4 QUANTITATIVE AND QUALITATIVE RESULTS

Quantitative. The char-CNN wins on every split (Table 3): **92.74%** (macro-F1 0.927, cross-entropy 0.238) on the Rosetta test split, **+2.60** over the baseline, and **82.84%** (macro-F1 0.822, loss 0.495) on the never-before-seen GitHub set, **+3.25** over it.

Table 3: All models, both test sets (accuracy %). “Held-out” = never-before-seen GitHub data. Chance 10%. Best in bold.

Model	Variant	Params	Val	Test (Rosetta)	Held-out (GitHub)
TF-IDF + NB (baseline)	raw	43,573 ft	91.80	90.14	79.59
TF-IDF + NB (baseline)	nocomment	41,472 ft	91.48	89.27	81.89
Char-BiGRU (rejected)	raw	42,186	92.59	90.27	80.54
Char-CNN (primary)	raw	68,938	94.31	92.74	82.84
Char-CNN (primary)	nocomment	68,938	94.69	92.49	82.16

Comment ablation. Stripping comments costs the CNN only **0.25** points (92.74→92.49): it reads *code structure, not comment prose*—what the two variants were built to test. The baseline behaves oppositely—removing comments *improves* its held-out score by 2.30 points (79.59→81.89)—so comment text misleads a bag-of-n-grams model on real code while the CNN is largely immune.

Qualitative. The 9.9-point gap between the two test sets (Figure 2) is the most interesting result so far, because the errors explain it. Two non-random modes dominate. (1) *Java→JavaScript* (30 errors): *licence headers*. **22 of the 30** contain an Apache/copyright header, and on average only **17% of their non-whitespace characters are code**—the rest is English prose in a `/* */` block. Rosetta files carry no licence headers, so the model never learned that a real file’s first 256 characters are often boilerplate identical across languages; it is starved of code, not wrong. (2) *PHP→Rust* (16 errors): *PHP 8 attributes*. **13 of the 16** contain `# [`, diagnosable straight from the training data: `# [` occurs in **0.0% of PHP training snippets (0/753)** but **9.0% of Rust (63/703)**, so the model learned the rule `# [⇒Rust`. PHP 8 spells attributes identically (`# [Group('...')]`) and they appear in **18.9% of held-out PHP (14/74)**, but Rosetta’s PHP predates PHP 8 so the construct is invisible in training. This is textbook *distribution shift*—confidently wrong for a legible reason—and PHP’s F1 collapses from 0.97 in-distribution to 0.66 held-out.

Challenges. (i) The held-out set took the most work: my first version yielded only 34 snippets/class because the C repositories I picked were too small, so I widened the list. (ii) The BiGRU’s 18× training cost made hyperparameter search impractical. (iii) MPS lacks deterministic kernels, so results move $\approx \pm 0.3$ points at a fixed seed; I treat sub-0.5-point gaps as noise. **On feasibility:** all data is obtained, the baseline is beaten on both test sets, and the pipeline reruns in under 3 minutes, so the work the error analysis points to—licence-header stripping (mode 1), post-2015 PHP sources (mode 2), and SCC (Alreshedy et al., 2018)—is comfortably achievable by the final report.

REFERENCES

- Rosetta code. <https://rosettacode.org>, 2026. Corpus mirror: <https://github.com/acmeism/RosettaCodeData>. Accessed: 2026-07-15.
- Kamel Alreshedy, Dhanush Dharmaretnam, Daniel M. German, Venkatesh Srinivasan, and T. Aaron Gulliver. SCC: Automatic classification of code snippets. *arXiv preprint arXiv:1809.07945*, 2018.
- Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of Sequences*, pp. 21–29. IEEE, 1997.
- GitHub. GitHub linguist. <https://github.com/github-linguist/linguist>, 2025. Accessed: 2026-07-15.
- Jyotiska Nath Khasnabish, Mitali Sodhi, Jayati Deshmukh, and Gopalakrishnan Srinivasaraghavan. Detecting programming language from source code using bayesian learning techniques. In *Machine Learning and Data Mining in Pattern Recognition (MLDM)*, pp. 513–522. Springer, 2014.
- Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1746–1751, 2014.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Secil Ugurel, Robert Krovetz, and C. Lee Giles. What’s the code? automatic classification of source code archives. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 632–638, 2002.
- Juriaan Kennedy van Dam and Vadim Zaytsev. Software language identification with natural language classifiers. In *IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, pp. 624–628, 2016.
- Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.